Atomically replaces the value stored in `*this` with the bitwise-or of the supplied value and the value stored in `*this` and returns the new value.

Declaration

```
integral-type operator|=(integral-type i) volatile noexcept;
integral-type operator|=(integral-type i) noexcept;
```

Effects

```
return this->fetch_or(i,std::memory_order_seq_cst) | i;
```

Std::Atomic<Integral-Type>::Operator-= Compound Assignment Operator

Atomically replaces the value stored in `*this` with the bitwise-or of the supplied value and the value stored in `*this` and returns the new value.

Declaration

```
integral-type operator^=(integral-type i) volatile noexcept;
integral-type operator^=(integral-type i) noexcept;
```

Effects

```
return this->fetch_xor(i,std::memory_order_seq_cst) ^ i;
```

Std::Atomic<T*> Partial Specialization

The `std::atomic<T*>` partial specialization of the `std::atomic` class template provides an atomic data type for each pointer type, with a comprehensive set of operations.

Instances of these `std::atomic<T*>` are not `CopyConstructible` or `CopyAssignable`, because these operations can’t be performed as a single atomic operation.

Class definition

```
template<typename T>
struct atomic<T*>
```